



VPC Peering

IR

irahimsindhu@gmail.com

The screenshot displays the AWS Management Console interface for a VPC peering connection. The main heading is "pcx-0b2896610eef4cd92 / VPC 1 <=> VPC 2". Below this, a status bar indicates "Pending acceptance" with a warning icon and a message: "You can accept or reject this peering connection request using the 'Actions' menu. You have until Sunday, June 21, 2026 at 01:32:21 GMT+5 to accept or reject the request, otherwise it expires." A modal dialog box titled "Accept VPC peering connection request" is open, asking "Are you sure you want to accept this VPC peering connection request? (pcx-0b2896610eef4cd92 / VPC 1 <=> VPC 2)". The dialog contains the following details:

Requester VPC	Accepter VPC	Requester CIDRs
vpc-00794030b273728e0 / NextWork-1-vpc	vpc-034734bfe2c626ef7 / NextWork-2-vpc	10.1.0.0/16
Requester Region	Accepter Region	
N. Virginia (us-east-1)	N. Virginia (us-east-1)	
Requester owner ID	Accepter owner ID	
666729139927(This account)	666729139927(This account)	

At the bottom of the dialog are "Cancel" and "Accept request" buttons. The background console shows the "Details" tab selected, with fields for "Requester owner ID", "Peering connection ID", "Status", and "Expiration time". Below the dialog, the "DNS settings" section is visible, showing "Requester VPC" and "Allow accepter VPC to resolve DNS of hosts in requester VPC to private IP addresses" (currently disabled).



Introducing Today's Project!

What is Amazon VPC?

Amazon VPC (Virtual Private Cloud) is a service that allows you to create and manage a private virtual network within AWS. It is useful because it gives you full control over your network configuration, including IP address ranges, subnets, route tables, and security settings, enabling you to securely deploy and manage AWS resources.

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to establish peering connection between 2 VPCs.

One thing I didn't expect in this project was...

One thing I didn't expect in this project was the simplicity in guidance.

This project took me...

This project took me 42 minutes.



In the first part of my project...

Step 1 - Set up my VPC

In this step, I will 2 separate VPCs using the resource map.

Step 2 - Create a Peering Connection

In this step, I will setup a peering connection between both of my VPCs. Peering connections allows private resources in VPCs to communicate with each other without traversing the public internet.

Step 3 - Update Route Tables

In this step, I will set up a way for the traffice coming from VPC1 to VPC2 and vice cersa. This step is important to ensure that data, resource etc can be shared easily.

Step 4 - Launch EC2 Instances

In this step, I will launch a dedicated EC2 instance in both of our VPCs.



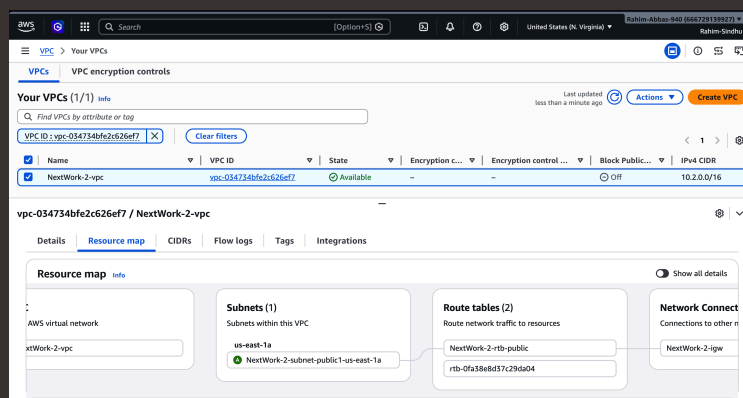
Multi-VPC Architecture

I started my project by launching 2 VPCs using the resource map. I set up 1 availability zone, 1 public subnet and 0 private subnet for my VPCs.

The CIDR blocks for VPCs 1 and 2 are 10.1.0.0/16 and 10.2.0.0/16. They have to be unique because we have to make sure that unique and distinct IPs are assigned to both VPCs. If it is not configured, the IPs will conflict with each other.

I also launched 2 EC2 instances

I didn't set up key pairs for these EC2 instances because AWS actually manages a key pair for us and we can access our EC2 instance using AWS EC2 Instance Connect.



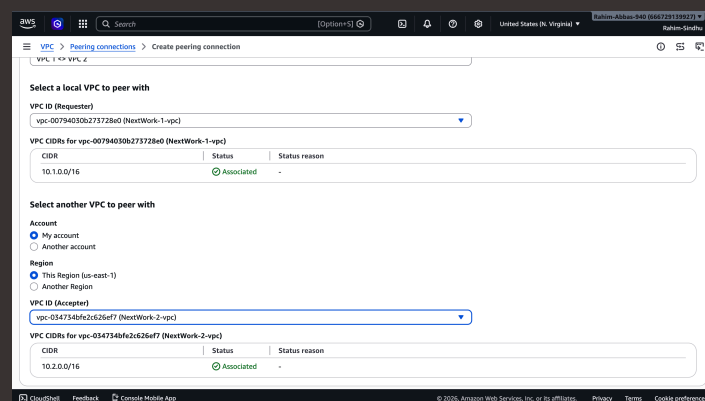


VPC Peering

A VPC peering connection is way through which 2 VPCs can communicate, share resources and other information using their IPs without every traversing the resource being shared using the public internet.

VPCs would use peering connections to separate their resources from public internet. Without a peering connection, VPCs will have to send the resources being shared first to the public internet and then to the other VPC.

The difference between a Requester and an Acceptor in a peering connection is that a requester will initiate the peering connection(in this project, VPC 1 is the requester) and an acceptor will accept the request and establish the peering connection.(In this project, VPC 2 is the accepter)





Updating route tables

After accepting a peering connection, my VPCs' route tables need to be updated because there still isn't a rule that dictates/guides the traffice from VPC1 to VPC2 and vice versa. Hence, we need to add a new route in the route tables of both VPC1 and VPC2.

My VPCs' new routes have a destination of the IPv4 CIDR block of the other VPC. The routes for VPC1 route table is: - Destination: 10.2.0.0/16 - Target: Peering Connection - Peering Connection: VPC1 <> VPC2 The routes for VPC2 route table is: - Destination: 10.1.0.0/16 - Target: Peering Connection - Peering Connection: VPC1 <> VPC2

The screenshot shows the AWS Management Console interface for a route table named `rtb-0a7d8d60478d5ffe1` in the `NextWork-2` VPC. A green notification banner at the top states: "Updated routes for rtb-0a7d8d60478d5ffe1 / NextWork-2-rtb-public successfully".

Details:

- Route table ID: `rtb-0a7d8d60478d5ffe1`
- VPC: `vpc-034734bfe2c626ef7` | `NextWork-2-vpc`
- Main: No
- Owner ID: `666729139927`
- Explicit subnet associations: `subnet-0d82c97fc4567fe` / `NextWork-2-subnet-public1-us-east-1a`
- Edge associations: -

Routes (3)

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	lgw-07dc3021317fd72b	Active	No	Create Route
10.1.0.0/16	pcx-0b2896610eef4cd92	Active	No	Create Route
10.2.0.0/16	local	Active	No	Create Route Table



In the second part of my project...

Step 5 - Use EC2 Instance Connect

In this step, I will Use EC2 Instance Connect to connect to your first EC2 instance.

Step 6 - Connect to EC2 Instance 1

In this step, I will Use EC2 Instance Connect to connect to Instance 1.

Step 7 - Test VPC Peering

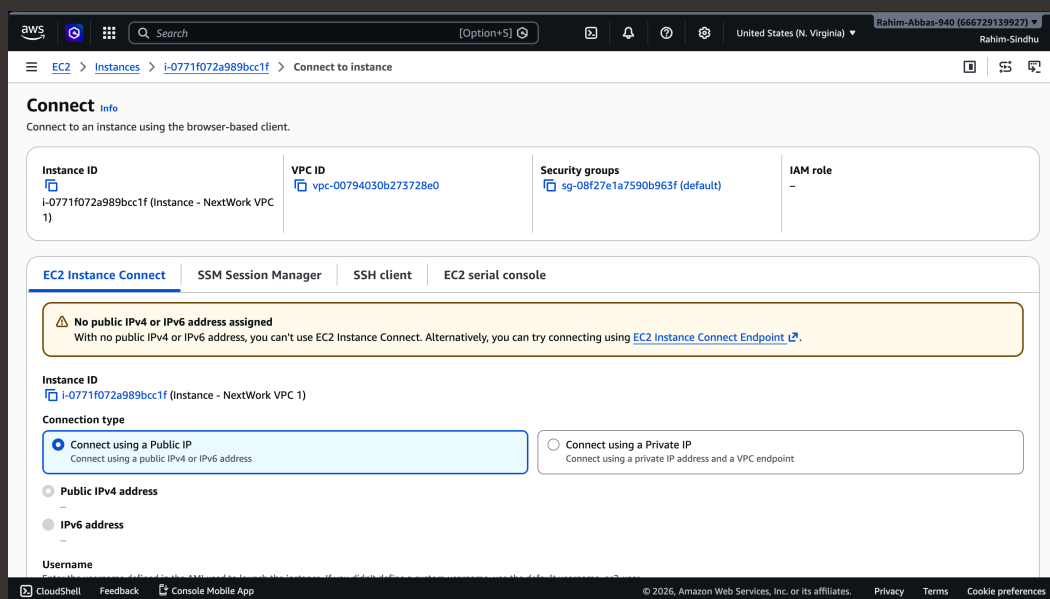
In this step, I will do the following: - Get Instance 1 to send test messages to Instance 2. - Solve connection errors until Instance 2 is able to send messages back.



Troubleshooting Instance Connect

EC2 Instance Connect is used to securely connect to an EC2 instance via SSH directly from the AWS Management Console, without needing to manually manage SSH keys on your local machine.

I was stopped from using EC2 Instance Connect as my instance did not have a IPv4 address. So, it resulted in an error.

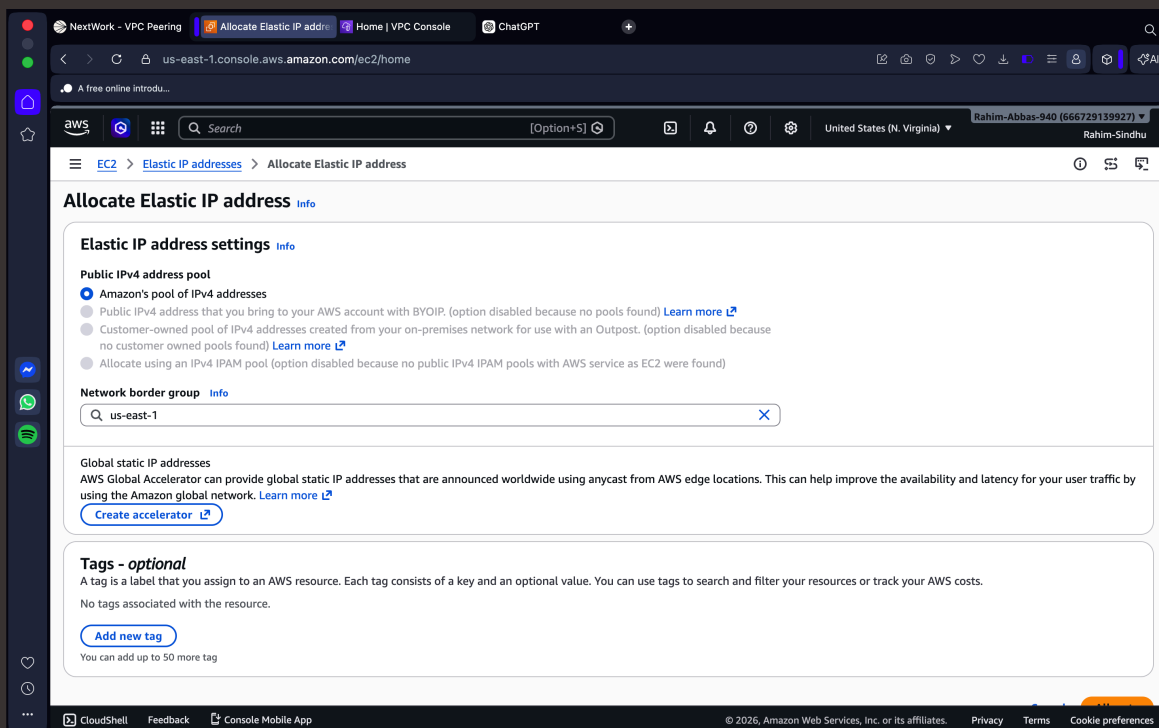




Elastic IP addresses

To resolve this error, I set up Elastic IP addresses. Elastic IP addresses are static public IP addresses that do not change dynamically. By associating an Elastic IP with my EC2 instance, the public IP and DNS mapping remained consistent, ensuring reliable connectivity even after the instance was stopped and started.

Elastic IP addresses solved the connection error by providing a permanent public IP address for the EC2 instance. Unlike standard public IP addresses, which can change when an instance is stopped and started, an Elastic IP remains the same. This ensured that the connection settings and DNS mappings stayed consistent, allowing reliable access to the instance.





Troubleshooting ping issues

To test VPC peering, I ran the command "ping" command.

A successful ping test would validate my VPC peering connection because ping command is used to test if the connection between 2 resources is established or not. If the connection is established, we get response in terms of packets with time, TTL and sequence number.

I had to update my second EC2 instance's security group because it was not allowing the traffic to reach it hence failing the ping command. I added a new rule that allowed the traffic from all ICMP IPv4 address.

```
[ec2-user@ip-10-1-12-101 ~]$ ping 10.2.9.242
PING 10.2.9.242 (10.2.9.242) 56(84) bytes of data:
64 bytes from 10.2.9.242: icmp_seq=1 ttl=127 time=0.209 ms
64 bytes from 10.2.9.242: icmp_seq=2 ttl=127 time=0.233 ms
64 bytes from 10.2.9.242: icmp_seq=3 ttl=127 time=0.229 ms
64 bytes from 10.2.9.242: icmp_seq=4 ttl=127 time=0.234 ms
64 bytes from 10.2.9.242: icmp_seq=5 ttl=127 time=0.251 ms
64 bytes from 10.2.9.242: icmp_seq=6 ttl=127 time=0.314 ms
64 bytes from 10.2.9.242: icmp_seq=7 ttl=127 time=0.228 ms
64 bytes from 10.2.9.242: icmp_seq=8 ttl=127 time=0.227 ms
64 bytes from 10.2.9.242: icmp_seq=9 ttl=127 time=0.231 ms
64 bytes from 10.2.9.242: icmp_seq=10 ttl=127 time=0.226 ms
64 bytes from 10.2.9.242: icmp_seq=11 ttl=127 time=0.253 ms
64 bytes from 10.2.9.242: icmp_seq=12 ttl=127 time=0.234 ms
64 bytes from 10.2.9.242: icmp_seq=13 ttl=127 time=0.227 ms
64 bytes from 10.2.9.242: icmp_seq=14 ttl=127 time=0.220 ms
64 bytes from 10.2.9.242: icmp_seq=15 ttl=127 time=0.340 ms
64 bytes from 10.2.9.242: icmp_seq=16 ttl=127 time=0.268 ms
64 bytes from 10.2.9.242: icmp_seq=17 ttl=127 time=0.225 ms
64 bytes from 10.2.9.242: icmp_seq=18 ttl=127 time=0.249 ms
64 bytes from 10.2.9.242: icmp_seq=19 ttl=127 time=0.260 ms
64 bytes from 10.2.9.242: icmp_seq=20 ttl=127 time=0.241 ms
64 bytes from 10.2.9.242: icmp_seq=21 ttl=127 time=0.244 ms
64 bytes from 10.2.9.242: icmp_seq=22 ttl=127 time=0.236 ms
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

